

上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

人工智能实践教学中心
Artificial Intelligence Practice Teaching Center

传感器与感知

蓝敏源
brucelan1777@sjtu.edu.cn

饮水思源 · 爱国荣校

目 录

01-基本调试与环境配置

您可以在此处添加您需要的文字，您可以在此处添加您需要的文字您可以在此处添加您需要的文字。

02-机器狗运动控制

您可以在此处添加您需要的文字，您可以在此处添加您需要的文字您可以在此处添加您需要的文字。

03-**传感器与感知**

您可以在此处添加您需要的文字，您可以在此处添加您需要的文字您可以在此处添加您需要的文字。

04-slam定位建图开发

您可以在此处添加您需要的文字，您可以在此处添加您需要的文字您可以在此处添加您需要的文字。

01

传感器与感知

核心内容：

① 机器狗简介：聚焦15自由度硬件、“去树莓派化”双控架构及核心传感器/机械臂功能，明确竞赛适配优势；

② 环境配置：实操完成设备开机校准、无线连接，搭建图形化/Python开发环境，调试通信链路与基础驱动，掌握设备日常维护要点。

01

机械臂控制

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

机械臂相关方法

设置机械臂末端位置arm(arm_x, arm_z)

参数名	格式	输入范围	说明
arm_x	float	[-80, 155]	单位为mm
arm_z	float	[-95, 155]	单位为mm

此处的x和z是相对于机械臂的基座的坐标，单位为毫米。

设定超过机械臂工作空间的值时，机械臂会保持最后一个有效值对应的姿态，比如(155,0)对应的姿态是向前伸到最大，(0,155)对应向上伸到最大，(155,155)是斜向上最大。当机械臂达不到预设位置时，就会保持上一次发送的有效位置。

机械臂相关方法

设置机械臂夹爪开合 claw(pos)

参数名	格式	输入范围	说明
pos	uint_8	0-255	0对应完全张开，255对应完全闭合

设置机械臂是否开启稳定模式 arm_mode(mode)

参数名	格式	输入范围	说明
mode	int	0\1	0不开启，1开启

开启之后机械臂末端会不随着身体的平移而平移（平移指四脚站定躯干运动，而非前后左右迈步平移）

机器狗运动开发

机械臂相关方法

创建一个类来封装机器狗机械臂的控制方法

预设机械臂的安全工作范围：

arm_x：-80 ~ 155 mm（水平面左右 / 前后移动极限）；

arm_z：-95 ~ 155 mm（垂直面上下移动极限，-95 为最低极限，对应桌面抓取）；

超出该范围时，不执行移动操作，仅打印错误提示，避免机械臂过载或卡死，保护硬件设备；

```python
from xgolib import XGO
import time

class RobotDogArm:
    def __init__(self):
        self.dog = XGO("xgolite")

    def move_arm(self, arm_x, arm_z):
        """
        控制机械臂移动到指定位置。

        :param arm_x: 水平面移动距离(mm)
        :param arm_z: 垂直面移动距离(mm)
        """
        # 检查输入值是否在机械臂工作范围内
        if -80 <= arm_x <= 155 and -95 <= arm_z <= 155:
            self.dog.arm(arm_x, arm_z)
        else:
            print("Error: Arm position out of range.")

    def control_claw(self, pos):
        """
        控制机械臂夹爪的开合。

        :param pos: 夹爪开合位置(0-255)
        """
        self.dog.claw(pos)
```

机器狗运动开发

机械臂相关方法

按顺序执行「移动机械臂→夹爪开到中间→夹爪完全张开→夹爪完全闭合」的一系列动作。

调用time.sleep(2)，暂停 2 秒，给机械臂足够时间完成移动动作（硬件动作响应慢于代码执行速度，延时是嵌入式硬件控制的必备操作）。

```
# 创建机器狗机械臂对象
my_arm = RobotDogArm()

# 移动机械臂到指定位置
my_arm.move_arm(20, 30)  # 例如：向前移动20mm，向上移动30mm

time.sleep(2)

# 控制夹爪开合
my_arm.control_claw(128)  # 例如：夹爪开合至中间位置
time.sleep(2)

# 夹爪完全张开
my_arm.control_claw(0)
time.sleep(2)

# 夹爪完全闭合
my_arm.control_claw(255)
time.sleep(2)
```

机器狗运动开发

机械臂相关方法

在控制机械臂时，使用异常处理来捕获并处理可能发生的错误。

常规方法可用try except捕获异常：

```python
# 尝试移动机械臂到超出工作范围的位置
try:
    my_arm.move_arm(200, 200)  # 这将超出机械臂的工作范围
except Exception as e:
    print(f"An error occurred: {e}")
```

可用条件判断先避免了出现错误的可能，也是程序控制中常见的一种避免出现错误的方法。

```python
def move_arm(self, arm_x, arm_z):
    """
    控制机械臂移动到指定位置。

    :param arm_x: 水平面移动距离(mm)
    :param arm_z: 垂直面移动距离(mm)
    """
    # 检查输入值是否在机械臂工作范围内
    if -80 <= arm_x <= 155 and -95 <= arm_z <= 155:
        self.dog.arm(arm_x, arm_z)
    else:
        print("Error: Arm position out of range.")
```

02

摄像头调用

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

摄像头功能开发

准备工作

在/home/pi/ 路径下创建文件夹，将camera1.py （摄像头调用包），以及需要运行的识别代码（如识别颜色color.py 或识别二维码 QR1.py)文件放入该文件夹中。

本节内容将针对摄像头调用，拍照，色彩识别，二维码识别，小球识别与抓取等几个方面具体展开。

camera1.py---摄像头调用，循环捕获图像、计算 FPS、标注并显示图像。

image.py---摄像头调用，拍照。

color.py---通过 HSV 颜色阈值分割、形态学去噪、轮廓检测，实现对红、绿、蓝三种颜色的实时识别。

QR1.py---通过将彩色帧转为灰度图提升识别效率，再借助pyzbar库完成二维码解析。

摄像头功能开发

image.py导入模块

```
from picamera2 import Picamera2  # 导入Picamera2库，用于控制摄像头
import time  # 导入时间模块，用于计时
import cv2 as cv  # 导入OpenCV库，用于图像处理
import numpy as np  # 导入numpy库，用于数组操作
```

关键说明：

Picamera2：仅支持树莓派系统（Raspberry Pi OS），替代旧版picamera，支持 Camera Module 2/3，捕获的图像直接返回numpy.ndarray，无需额外格式转换；

cv2：这里主要用到 3 个功能 ——cv.putText()（文字标注）、cv.imencode()（JPG 编码）、cv.imshow()（窗口显示）

摄像头功能开发

类的构造函数----创建相机实例时自动执行，负责初始化摄像头硬件和参数

create_preview_configuration：创建「预览模式」配置，适合实时图像捕获（如视频流、追踪），兼顾速度和清晰度，区别于「拍照模式」（create_still_configuration，追求高分辨率）；

图像格式RGB888：OpenCV 默认处理的是 BGR 格式，但这里标注文字、编码 JPG 都无需转换，因为cv.putText()和cv.imencode()对 RGB 格式图像同样兼容，不会出现颜色错乱。

```
class Dog_Camera(object):
    """狗狗相机类，用于控制摄像头进行图像捕获和处理"""
    def __init__(self, width=640, height=480, debug=False):
        self.__debug = debug  # 调试模式标志（True时输出调试信息）
        self.__state = False  # 相机状态标志（True表示开启，False表示关闭）
        self.__width = width  # 捕获图像的宽度
        self.__height = height  # 捕获图像的高度
        print("---------Camera Init...------------")  # 打印初始化提示

        # 初始化相机对象
        self.__picam2 = Picamera2()
        # 创建预览配置：设置图像格式为RGB888，尺寸为指定的宽高
        config = self.__picam2.create_preview_configuration(main={"format": 'RGB888', "size": (self.__width, self.__height)})
        self.__picam2.configure(config)  # 应用配置
        self.__picam2.start()  # 启动相机

        self.__state = True  # 更新相机状态为开启

        if self.__debug:  # 若开启调试模式，打印初始化完成信息
            print("---------Camera Init OK!------------")
```

摄像头功能开发

get_frame()（获取原始 RGB 图像帧）

```
def get_frame(self):
    if self.__state:
        frame = self.__picam2.capture_array()
        return True, frame
    else:
        return False, None
```

获取摄像头实时捕获的原始图像，返回 numpy 数组格式，供后续处理（如目标识别、图像增强）。

核心方法capture_array()：Picamera2 获取实时图像的核心接口，效率极高，返回的数组形状为(height, width, 3)（对应高、宽、RGB 三通道）；

摄像头功能开发

get_frame_jpg()（获取 JPG 编码字节流）

```python
def get_frame_jpg(self, text="", color=(0, 255, 0)):
    success, image = self.get_frame()
    if not success:
        return success, bytes({1})

    if text != "":
        cv.putText(image, str(text), (10, 20), cv.FONT_HERSHEY_SIMPLEX, 0.5, color, 1)

    success, jpeg = cv.imencode('.jpg', image)
    return success, jpeg.tobytes()
```

在get_frame()的基础上，增加文字标注功能，并将图像编码为 JPG 格式的字节流，适合网络传输（如网页推流）、文件保存。

文字标注cv.putText()：参数color是 BGR 格式（默认绿色(0,255,0)），即使图像是 RGB 格式，标注的文字颜色也会正常显示，无需转换；文字坐标(10,20)表示图像左上角向右 10 像素、向下 20 像素，避免文字超出图像边界；

摄像头功能开发

主程序测试（if __name__ == '__main__':）

创建相机实例，循环捕获图像、计算 FPS、标注并显示图像。

FPS 计算：

即时 FPS：适合观察实时性能，每帧耗时越短，FPS 越高，波动较大（受硬件性能、系统负载影响）；

平均 FPS：适合评估整体性能，总帧数除以总耗时，波动较小；

```python
if __name__ == '__main__':
    # 测试代码：创建相机实例（开启调试模式）
    camera = Dog_Camera(debug=True)
    average = False  # 是否计算平均FPS（False为即时FPS）
    m_fps = 0  # 帧计数器（用于平均FPS计算）
    t_start = time.time()  # 计时起点（用于平均FPS计算）

    # 循环获取并显示图像，直到相机关闭
    while camera.isOpened():
        if average:  # 计算平均FPS
            ret, frame = camera.get_frame()  # 获取图像帧
            m_fps += 1  # 累加帧数
            fps = m_fps / (time.time() - t_start)  # 平均FPS = 总帧数 / 总时间
        else:  # 计算即时FPS
            start = time.time()  # 记录当前时间
            ret, frame = camera.get_frame()  # 获取图像帧
            end = time.time()  # 记录获取完成时间
            fps = 1 / (end - start)  # 即时FPS = 1 / 单帧耗时

        # 在图像上绘制FPS信息：内容、位置、字体、大小、颜色、线条粗细
        text = "FPS:" + str(int(fps))
        cv.putText(frame, text, (20, 30), cv.FONT_HERSHEY_SIMPLEX, 0.9, (0, 200, 0), 1)

        cv.imshow('frame', frame)  # 显示图像窗口

        # 等待按键输入（1毫秒），按ESC(27)或q键退出循环
        k = cv.waitKey(1) & 0xFF
        if k == 27 or k == ord('q'):
            break

    del camera  # 释放相机资源
    cv.destroyAllWindows()  # 关闭所有OpenCV窗口
```

03

目标追踪

您可以在此处添加您需要的文字，您可以在此处添加您需要的。

目标识别

实验目标

- 学习如何控制MAX机器人的摄像头。
- 掌握使用视觉模型进行手势识别的方法。
- 完成小球追踪实验

在这一节，我们引入一个新的python包：xgoedu 教育库。

XGO的教育库主要是给出了集成在AI模组中的摄像头，屏幕，按键，麦克风和喇叭等硬件的Python接口。以及部分AI模型调用等功能。

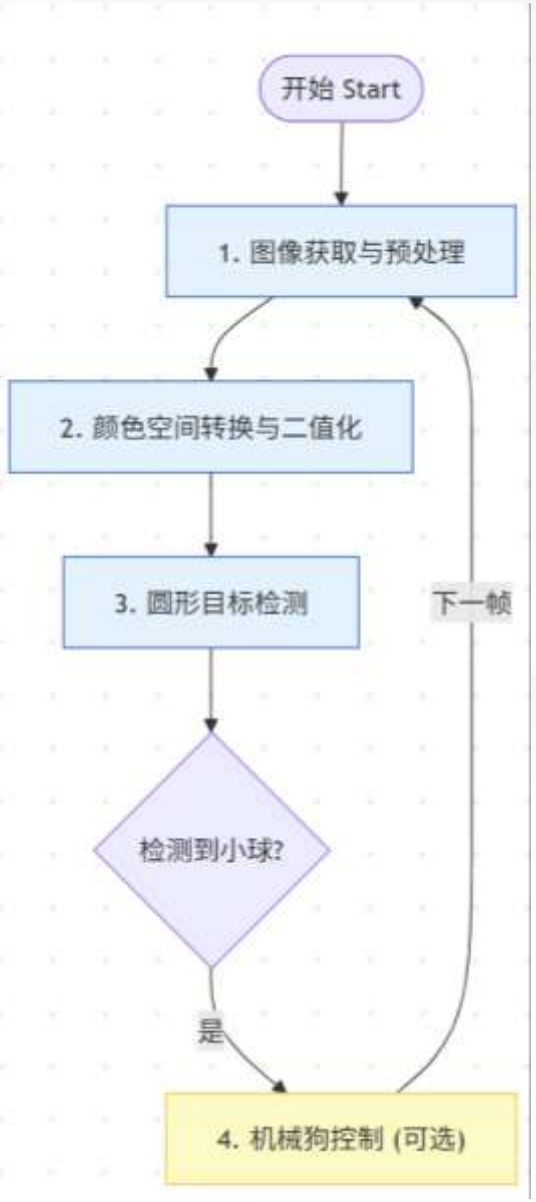
```
1    # 创建XGO教育库的实例
2    XGO_edu = XGOEDU()
```

目标识别

文字与图片显示

颜色识别原理：颜色识别原理：利用自定义的 detect_red_ball_on_table() 函数，通过设置HSV色彩空间中红色的两个范围（低区0-15和高区160-180），对摄像头获取的图像进行二值化和遮罩处理，识别出红色小球。首先通过 cv2.inRange() 结合两个红色范围创建颜色掩码，并进行形态学开闭操作去除噪声；然后对遮罩后的图像转换为灰度图并使用高斯模糊；最后使用霍夫圆变换 cv2.HoughCircles() 在颜色掩码区域内检测圆形，找到最大的圆作为目标，返回其圆心坐标和半径。

追踪控制原理：根据颜色识别得到的小球位置信息，判断小球在图像中的位置偏差。通过调整机器人的运动方向和速度，使小球保持在摄像头的视野中心，实现追踪功能。当小球在左侧时，机器人左转；在右侧时，机器人右转；**距离较近时，机器人减速或停止；距离较远时，机器人前进。**



目标识别

实验准备与步骤

亮度均衡化：转换BGR到LAB空间（cv2.cvtColor），对L通道做自适应直方图均衡（cv2.createCLAHE）

多通道阈值分割：颜色二值化（binarize_in_lab()），分离LAB三通道（cv2.split）

联合阈值过滤（cv2.inRange+cv2.bitwise_and）

形态学+轮廓分析：目标提取（find_largest_circle()）：开闭运算降噪（cv2.morphologyEx），找最大轮廓拟合圆（cv2.findContours+cv2.minEnclosingCircle）

颜色识别的部分只需要同学们调整色彩阈值。

在LAB_Helper中调整色彩阈值。选出我们希望的色彩之后填入色彩阈值当中。



目标识别

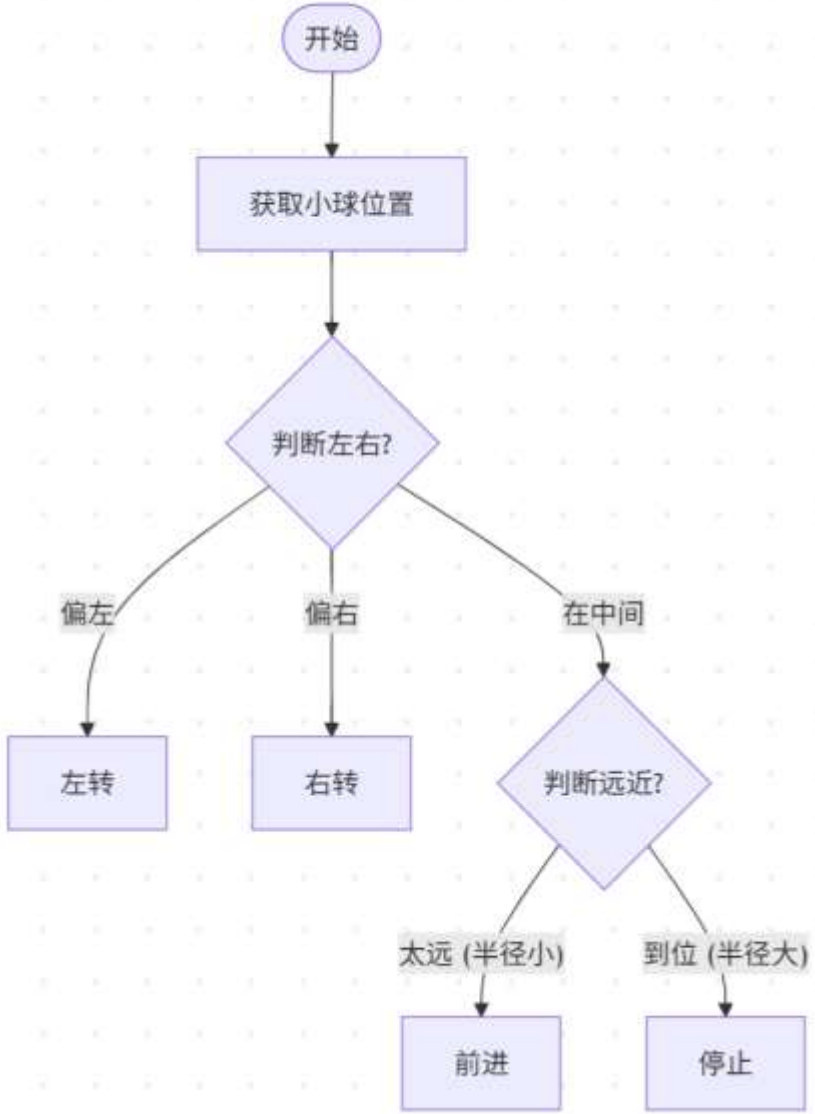
追踪实验设计

集成颜色识别和运动控制，实现小球追踪：

在识别小球的循环中，根据小球位置调用机器人的 move 或 turn 方法。

例如：当小球在左侧（x 坐标小于图像中心），机器人左转；在右侧则右转。

实时显示追踪效果，调整追踪参数（如旋转速度、阈值等）以优化追踪性能。



目标识别

追踪算法实现

根据小球位置坐标，设计追踪逻辑：

计算小球在图像中的水平偏移量（x 坐标与图像中心 x 坐标的差值）。当偏移量大于阈值时，控制机器人向相反方向旋转，使小球回到视野中心。

结合小球大小或距离信息，控制机器人的前进或后退，保持合适的追踪距离。

```python
def keep_center(ball: BallInfo):
    """控制机器狗调整姿态（yaw左右转、pitch俯仰），保持小球在画面中心"""
    # 配置参数：根据机器狗硬件调整
    max_yaw = 11     # 最大左右旋转角度（°）
    max_pitch = 15   # 最大上下俯仰角度（°）
    gain = 1.0       # 控制增益系数，值越大反应越快（过大会抖动）

    # 1. 将归一化位置转换为控制偏移量
    # x方向：小球偏右（x>0）→ 机器狗左转（负yaw）；小球偏左（x<0）→ 机器狗右转（正yaw）
    yaw_offset = -ball.x * max_yaw * gain
    # y方向：小球偏上（y>0）→ 机器狗抬头（正pitch）；小球偏下（y<0）→ 机器狗低头（负pitch）
    pitch_offset = ball.y * max_pitch * gain

    # 2. 限幅：防止偏移量超过机器狗最大运动范围，避免硬件损坏
    yaw_offset = np.clip(yaw_offset, -max_yaw, max_yaw)
    pitch_offset = np.clip(pitch_offset, -max_pitch, max_pitch)

    # 3. 发送控制命令（连接机器狗则执行，否则仅模拟）
    try:
        if dog is not None:
            # 控制机器狗调整yaw（左右）和pitch（上下）姿态
            dog.attitude(['y', 'p'], [yaw_offset, pitch_offset])
            print(f"【硬件控制】yaw={yaw_offset:.2f}°, pitch={pitch_offset:.2f}°")
        else:
            print(f"【模拟控制】yaw={yaw_offset:.2f}°, pitch={pitch_offset:.2f}°（未连接机器狗）")
    except Exception as e:
        print(f"控制命令发送失败：{e}")
```

实验总结

本实验实现了机器人对小球的颜色识别和追踪功能。通过颜色识别获取小球位置，结合运动控制使机器人能够自动追踪小球，提高了机器人的环境感知和自主运动能力。在实验中，颜色遮罩的获取精度和追踪算法的参数调整是关键，需要多次测试以适应不同的光照条件和小球颜色。同时，也发现了机器人追踪速度和精度之间的矛盾，需要进一步优化算法来平衡两者。

思考题 / 课后作业：

1. 4.尝试识别多种颜色的小球，并根据不同颜色执行不同的追踪策略。